Système d'exploitation : TD6

mars 2026 — Pierre Rousselin

Dans ce TD, on va voir comment lire des octets dans un fichier avec l'appel système `read` et interpréter ces octets.

Voici le prototype de `read` :

```
/* Lire au plus count octet depuis le descripteur de fichier fd et les recopier
 * dans le tableau buf. La tête de lecture sur le fichier avance autant que
 * le nombre d'octets lus.
 * Retourne :
 * - le nombre d'octets effectivement lus en cas de succès
 * - 0 si l'on atteint la fin du fichier
 * - -1 en cas d'erreur */
ssize_t read(int fd, void *buf, size_t count);
```

— L'entier `fd` est un *descripteur de fichier* ouvert par le processus. Par exemple, 0 décrit l'entrée standard.

  On ne se souciera pas dans ce TD de l'ouverture des fichiers.

— Le tableau d'adresse `buf` *doit* pouvoir contenir au moins `count` octets.

— Le type `ssize_t` est un type entier signé destiné à accueillir des valeurs positives assez grandes et au minimum -1 pour signaler une éventuelle erreur. On supposera dans ce TD que ces erreurs n'arrivent pas, sauf indication contraire.

Exercice 1 :

Implémenter les fonctions spécifiées ci-dessous.

1.
```
#define EOF -1
/* Retourner le prochain caractère du fichier de descripteur fd,
 * ou EOF si on a atteint la fin du fichier. */
int mon_getc(int fd);
```

Pourquoi avoir retourné un `int` et pas un `char` ? (Remarque : nous faisons ici le même choix que dans la fonction standard `getchar`).

2.
```
/* Retourner le prochain caractère non blanc du fichier de
 * descripteur fd, ou EOF si on a atteint la fin du fichier. */
int mon_get_nonblank_c(int fd);
```

Remarque : on implémente ici une fonction similaire à `scanf(" %c", &c)`

3.
```
/* Lire une ligne depuis le fichier de descripteur de fichier fd
 * et la placer dans le tampon pointé par buf jusqu'à atteindre un
 * retour chariot ou la fin du fichier remplacé par un octet nul. */
void mon_gets(int fd, char *buf);
```

Il est écrit dans le manuel de `gets` :

```
DESCRIPTION
    Ne jamais utiliser cette fonction.
```

Pourquoi, à votre avis ?

4.
```
/* Lire au plus size - 1 caractère dans le fichier de descripteur
 * de fichier fd et les stocker dans le tampon d'adresse buf. La
 * lecture s'arrête après une fin de fichier ou un retour chariot.
 * Si un retour chariot est lu, il est stocké dans buf.
 * Retourne buf en cas de succès, NULL en cas d'erreur, ou si la fin
 * du fichier est atteinte sans que des caractères aient été lus. */
char *mon_fgets(int fd, char *buf, size_t size);
```

5.
```
/* Met à l'adresse n la valeur de l'entier non signé représenté
 * sous forme décimale dans le fichier de descripteur fd. La lecture
 * de l'entier commence au première caractère non blanc rencontré
```

```
 * et s'arrête au premier caractère non décimal rencontré,
 * ou à la fin du fichier.
 * Retourne 1 si un entier naturel a bien été lu, EOF si la fin du
 * fichier est atteinte avant la lecture d'un entier et 0 si
 * aucun chiffre décimal n'a été rencontré avant la lecture d'un
 * caractère non décimal. */
int lire_nat(int fd, unsigned *n);
```

Remarque : on implémente ici une fonction similaire à `scanf("%u", &n)`, mais il y a tout de même un petit problème (lequel ?)

6.
```
/* Met à l'adresse n la valeur de l'entier non signé correspondant
 * aux 4 prochains octets (dans le même boutisme) dans le fichier
 * de descripteur fd.
 * On suppose que ce fichier n'est qu'une suite d'entiers non signés
 * sur 4 octets dans le même boutisme que la machine.
 * Retourne 1 si un entier naturel a bien été lu, EOF si la fin du
 * fichier est atteinte. */
int lire_nat_bin(int fd, unsigned *n);
```

--- * ---

Exercice 2 : Vers les *buffers*

On souhaite implémenter ici un tampon pour la lecture sur le terminal (un tel tampon serait utile pour n'importe quel fichier sans changement majeur). Voici quelques définitions et déclarations en C.

```
#define BUFSZ 1024
#define EOF -1
struct flux_stdin {
        char *buffer;
        size_t bufsz;
        char *last_read_char;
        char *next_char;
        int eof;
};
struct flux_stdin _flux_stdin = { NULL, BUFSZ, NULL, NULL, 0 };
struct flux_stdin *stdin = &_flux_stdin;
int getchar(void);
int ungetchar(int c);
```

Le but est d'utiliser `getchar` comme primitive pour toutes les opérations de lecture sur l'entrée standard et de faire peu d'opérations `read` en utilisant le pointeur global `stdin`. Les pointeurs `last_read_char` et `next_char` devraient pointer, respectivement vers le dernier caractère lu avec `read` dans le buffer et le prochain caractère à retourner avec `getchar`. Le booléen `eof` vaut `1` si la fin du fichier a été atteinte.

1. Lors d'un appel à `getchar`, si `buffer` n'a pas encore été alloué, on commence par l'allouer (`bufsz` caractères sur le tas).

2. Si `last_read_char` est `NULL` ou inférieur à `nextchar`, on fait un nouveau `read`, aussi gros que possible, et on repositionne `last_read_char` et `next_char`.

3. On retourne le caractère pointé par `next_char` et on fait avancer `next_char`.

4. Maintenant, réfléchir à l'implémentation de `ungetchar`.

5. Quel est l'avantage d'utiliser notre `getchar` par rapport aux `read` de 1 octet du premier exercice ?

6. Résoudre le problème de la fonction `lire_nat` avec `ungetchar`.

--- * ---